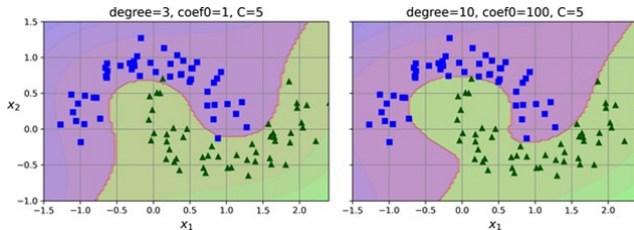


2.5 Kernel Đa thức (Polynomial Kernel)

- 'SVC(kernel="poly", degree=3)'
- Tham số 'coef0': Kiểm soát mức độ ảnh hưởng của đa thức bậc cao so với đa thức bậc thấp.
- Hình ảnh cho thấy sự khác biệt giữa mô hình bậc 3 (trái) và bậc 10 (phải).



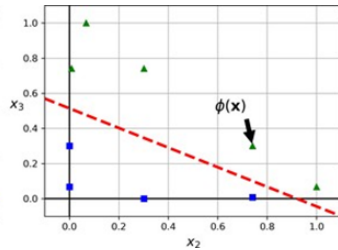
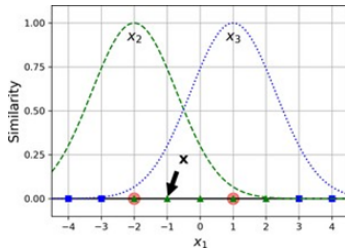
Hình 5-7: Polynomial Kernel với bậc 3 và 10

2.6 Đặc trưng tương tự (Similarity Features)

- Một kỹ thuật khác: Bổ sung các đặc trưng sử dụng **Hàm tương tự (Similarity Function)**, đo lường mỗi mẫu giống các Mốc (Landmarks) đến mức nào.
- Ví dụ: Gaussian Radial Basis Function (RBF).
- Công thức: $\phi_{\gamma}(x, \ell) = \exp(-\gamma \|x - \ell\|^2)$
- Giá trị này dạng hình chuông, thay đổi từ 0 (rất xa) đến 1 (ngay tại mốc).

2.7 Minh họa Hàm Gaussian RBF

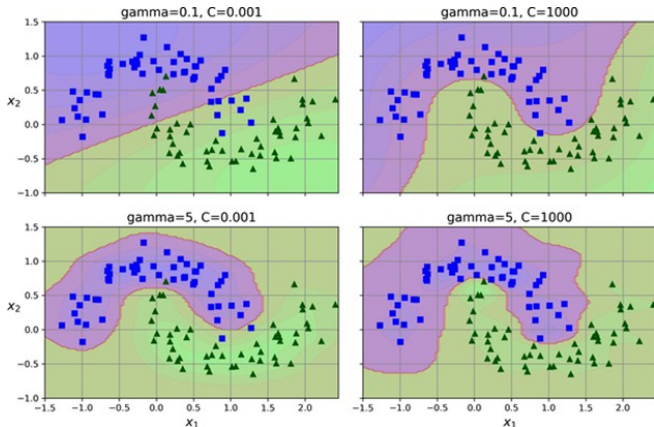
- Đặt mốc tại $x_1 = -2$ và $x_1 = 1$.
- Chuyển điểm $x = -1$ thành 2 chiều mới: khoảng cách tới mốc 1 và khoảng cách tới mốc 2.
- Giờ đây tập dữ liệu đã có thể phân tách tuyến tính bằng 1 đường thẳng.



Hình 5-8: Sử dụng Similarity Features phân tách dữ liệu 1D

2.8 Gaussian RBF Kernel

- Giống đa thức, ta dùng **Kernel RBF** để hưởng lợi ích của vô vàn các Mốc (Landmarks) mà không phải tính toán chi phí cao.
- 'SVC(kernel="rbf", gamma=5, C=0.001)'
- Tham số 'gamma' (γ): Định hình độ hẹp/rộng của quả chuông.



Hình 5-9: Hiệu ứng của γ và C lên SVM Kernel RBF

2.9 Điều chỉnh siêu tham số Gamma và C

- **Gamma (γ) lớn:** Quả chuông nhọn hẹp. Ranh giới uốn lượn sát theo từng điểm dữ liệu (Overfitting).
- **Gamma (γ) nhỏ:** Quả chuông rộng, mượt. Ranh giới mượt mà hơn (Underfitting).
- Lời khuyên: Nếu mô hình bị Overfitting, hãy giảm γ . Ngược lại, hãy tăng γ . (Tương tự như C).
- Rule of thumb: Khi nghi ngờ, hãy luôn thử **Linear Kernel** trước, sau đó tới **Gaussian RBF Kernel**.

3.1 Các Lớp của Scikit-Learn cho SVM

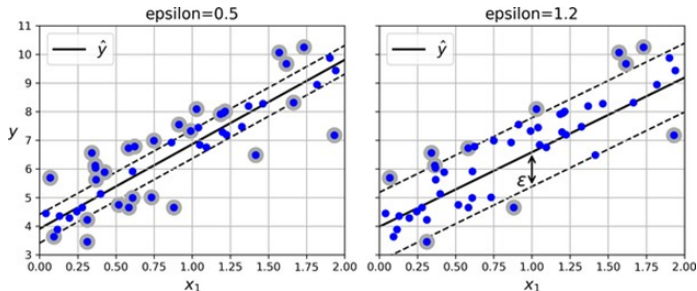
- 'LinearSVC': Dùng thư viện LIBLINEAR, tối ưu cho tập dữ liệu cực lớn, phức tạp thời gian $\mathcal{O}(m \times n)$. (Chỉ chạy tuyến tính).
- 'SVC': Dùng thư viện LIBSVM, hỗ trợ Kernel trick. Phức tạp thời gian $\mathcal{O}(m^2 \times n)$ đến $\mathcal{O}(m^3 \times n)$. (Quá chậm nếu số lượng mẫu $> 100,000$).
- 'SGDClassifier(loss="hinge")': Huấn luyện Gradient Descent ngẫu nhiên, hữu ích cho tập dữ liệu khổng lồ (Out-of-core learning).

3.2 Ý tưởng Hồi quy bằng SVM

- Thuật toán SVM cực kỳ đa năng. Nó không chỉ phân loại tốt mà còn hồi quy (Dự đoán giá trị số liên tục) xuất sắc!
- Trong **Phân loại SVM**, mục tiêu là để dải lề rộng nhất nhưng KHÔNG chứa bất cứ điểm vi phạm nào bên trong dải lề.
- Ngược lại, trong **Hồi quy SVM**, mục tiêu là đưa CÀNG NHIỀU ĐIỂM VÀO BÊN TRONG dải lề càng tốt, trong khi hạn chế số điểm rơi ra ngoài dải lề.
- Độ rộng lề được kiểm soát bởi siêu tham số Epsilon (ϵ).

3.3 Minh họa SVM Regression Tuyến tính

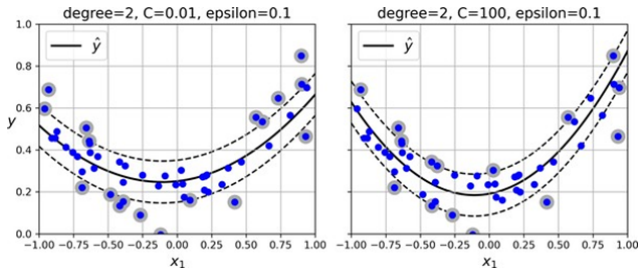
- Hình bên minh họa 'LinearSVR'.
- Việc thêm nhiều điểm dữ liệu nằm bên trong biên không làm ảnh hưởng đến dự đoán của mô hình.
- Mô hình gọi là ϵ -insensitive.
- ϵ lớn (trái) lề rộng. ϵ nhỏ (phải) lề hẹp.



Hình 5-10: Hồi quy LinearSVR với các giá trị ϵ khác nhau

3.4 Hồi quy SVM Phi tuyến (Nonlinear SVR)

- Hồi quy SVM cũng hỗ trợ Kernel trick thông qua lớp 'SVR(kernel="poly")'.
- Trong hình là dữ liệu theo dạng hàm bậc 2. Mô hình có thể bám rất sát xu hướng với đường cong mịn màng.
- C nhỏ (trái): Mô hình tự do, linh hoạt.
- C lớn (phải): Ít sai số, sát dữ liệu hơn (ít regularization).



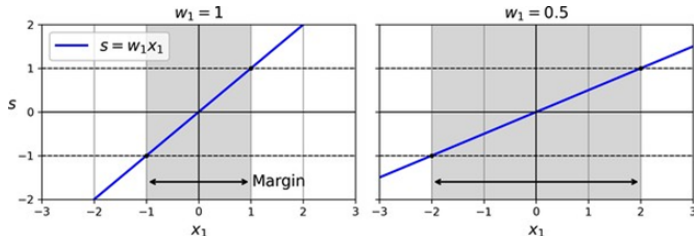
Hình 5-11: Hồi quy SVM phi tuyến bằng Kernel Đa thức

4.1 Hàm Quyết định (Decision Function)

- SVM tuyến tính dự đoán bằng cách tính hàm quyết định (Decision Function):
- Hàm số: $f(\mathbf{x}) = \mathbf{w}^T \cdot \mathbf{x} + b = w_1x_1 + \dots + w_nx_n + b$
- Nếu kết quả dương, dự đoán lớp 1 (Tích cực). Nếu âm, dự đoán lớp 0 (Tiêu cực).
- $\hat{y} = 1$ if $\mathbf{w}^T \mathbf{x} + b \geq 0$, else 0.

4.2 Đồ thị Không gian Hàm quyết định

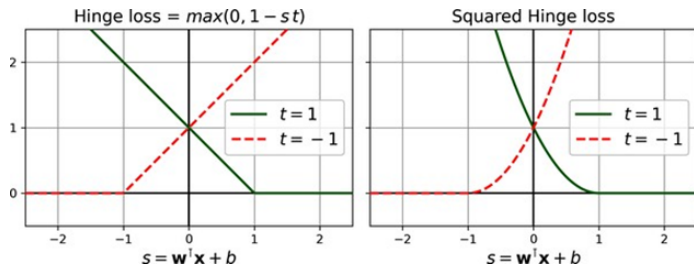
- Không gian 3 chiều: Trục Z là giá trị của hàm quyết định.
- Mặt phẳng cắt tại $Z = 0$ tạo thành ranh giới quyết định (Đường liền đen).
- Đường lề đứt nét tương ứng với mặt cắt $Z = 1$ và $Z = -1$.



Hình 5-12: Decision function cho mô hình Iris
Dataset 2D

4.3 Mối liên hệ giữa Vector Trọng số ($\mathbf{w}$) và Biên

- Kích thước của véc-tơ trọng số $\mathbf{w}$ xác định độ dốc của mặt phẳng hàm.
- Nếu chia đôi độ dốc $\mathbf{w}$, độ rộng của lề (Margin) sẽ nhân đôi!
- Để có Margin cực đại, ta phải **tối thiểu hóa độ lớn của $\mathbf{w}$** , tức là $\|\mathbf{w}\|$.



Hình 5-13: Trọng số $\mathbf{w}$ càng nhỏ, Lề margin càng lớn

4.4 Hàm Mục Tiêu của Phân loại Biên Cứng

- Ta muốn tối thiểu hóa $\|\mathbf{w}\|$ (để lề rộng nhất).
- Điều kiện ràng buộc: Tất cả mẫu lớp 1 phải có $f(\mathbf{x}) \geq 1$, và lớp 0 phải có $f(\mathbf{x}) \leq -1$.
- Đặt $t^{(i)} = 1$ nếu $y^{(i)} = 1$ và $t^{(i)} = -1$ nếu $y^{(i)} = 0$. Ràng buộc gộp lại thành:
 $t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1$ với mọi mẫu.
- Bài toán tối ưu hóa có ràng buộc:
- Cực tiểu hóa: $\frac{1}{2} \mathbf{w}^T \mathbf{w}$
- Sao cho: $t^{(i)}(\mathbf{w}^T \mathbf{x}^{(i)} + b) \geq 1 \quad \forall i$

4.5 Hàm Mục Tiêu của Phân loại Biên Mềm

- Trong Biên mềm, ta chấp nhận lỗi. Ta định nghĩa biến Slack $\zeta^{(i)} \geq 0$.
- $\zeta^{(i)}$ đo lường mức độ mẫu thứ i được phép vi phạm lề.
- Tối thiểu hóa 2 mục tiêu mâu thuẫn nhau: làm cho $\mathbf{w}$ nhỏ nhất và làm cho tổng ζ nhỏ nhất.
- Bài toán tối ưu: Cực tiểu hóa $\frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^m \zeta^{(i)}$
- Siêu tham số C xác định sự đánh đổi giữa hai mục tiêu này.

4.6 Trình giải Bài toán (Solvers)

- Đây là các bài toán **Quy hoạch Toàn phương (Quadratic Programming - QP)** có lời giải tối ưu toàn cục.
- Có rất nhiều solver có sẵn để giải: SMO, LIBLINEAR, LIBSVM.
- Có một dạng khác: Dùng Gradient Descent thông thường để cực tiểu hóa Hinge Loss. Đó chính là 'SGDClassifier'.